9e. Reductions

Martin Alfaro
PhD in Economics

INTRODUCTION

Reductions refer to **operations that take a collection as their input and return a single element as their output**. Such operations arise naturally in a wide range of contexts, with summary statistics as a typical example (e.g., averages, variances, or maxima of collections).

The technique involves iteratively applying an operation to pairs of elements, accumulating the result at each step until the final output is obtained. A classic example of reduction is the summation of all numeric elements in a vector. This arrives at the final result by applying the addition operator `+` to pairs of elements, iteratively updating the accumulated sum. The technique applied to this case is illustrated below.

OPERATION

```
x = rand(100)

foo(x) = sum(x)
```

```
julia> foo(x)
48.447
```

IMPLEMENTATION VIA REDUCTION

```
x = rand(100)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output = output + x[i]
    end

    return output
end
```

```
julia> foo(x)
48.447
```

IMPLEMENTATION VIA REDUCTION (EQUIVALENT)

```
x = rand(100)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output += x[i]
    end

    return output
end
```

```
julia> foo(x)
48.447
```

The last tab implements the reduction via an **update operator**. These operators are frequently used in reductions because they streamline notation, rewriting expressions like `x = x + a` in the more compact form `x += a`.

The main benefit of reductions is that, by operating on scalars, **they don't create memory allocations**. This is particularly convenient when a vector must be transformed prior to aggregating the result. For instance, if you need to compute `sum(log.(x))`, the procedure would avoid the allocations of the intermediate vector `log.(x)`.

IMPLEMENTING REDUCTIONS

Technically, reductions are implemented by iteratively applying a *binary operation* to pairs of values, ultimately producing a scalar result. For the procedure to be valid, the binary operation must satisfy **two mathematical properties**:

- **Associativity**: the grouping of operations doesn't affect the outcome. For example, scalar addition is associative because $(a + b) + c = a + (b + c)$.
- **Existence of an identity element**: there exists an element that, when combined with any other element through a binary operation, leaves that element unchanged. For example, the identity element of scalar addition is 0 because $a + 0 = a$.

The identity element serves as the initial value of the accumulator variable. [1] Each operation has its own identity element, as shown in the table below.

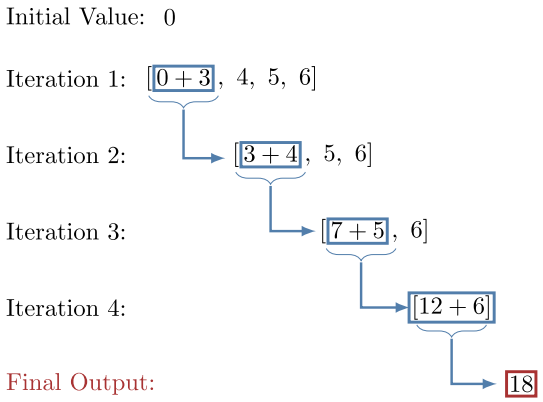
Operation	Identity Element
Sum	0
Product	1
Maximum	-Inf

Operation	Identity Element
Minimum	Inf

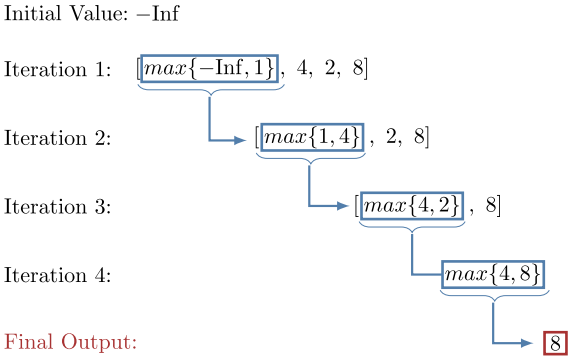
Remarkably, binary operations can be expressed either as **binary operators** or as two-argument functions. For instance, the symbol `+` can be employed in either form, making `output = output + x[i]` equivalent to `output = +(output, x[i])`. The possibility of using functions for reductions expands their scope. For instance, it enables the application of the `max` function to find the maximum value in a vector `x`, where `max(a,b)` returns the larger of the two scalars `a` and `b`.

The figures below visually illustrate how a reduction operates. They describe the process both when expressed with a binary operator and with a two-argument function.

REDUCTION via OPERATOR: sum of [3,4,5,6]



REDUCTION via FUNCTION: maximum of [1,4,2,8]



To implement reductions manually, we typically employ for-loops. To illustrate this formulation, we introduce `foo1` to identify the desired outcome, with `foo2` providing the same result through a reduction.

SUM

```
x       = rand(100)

foo1(x) = sum(x)

function foo2(x)
    output = 0.0

    for i in eachindex(x)
        output += x[i]
    end

    return output
end
```

```
julia> foo1(x)
48.447
julia> foo2(x)
48.447
```

PRODUCT

```
x       = rand(100)

foo1(x) = prod(x)

function foo2(x)
    output = 1.0

    for i in eachindex(x)
        output *= x[i]
    end

    return output
end
```

```
julia> foo1(x)
1.80982e-43
julia> foo2(x)
1.80982e-43
```

MAXIMUM

```
x       = rand(100)

foo1(x) = maximum(x)

function foo2(x)
    output = -Inf

    for i in eachindex(x)
        output = max(output, x[i])
    end

    return output
end
```

```
julia> foo1(x)
0.991405
julia> foo2(x)
0.991405
```

MINIMUM

```
x       = rand(100)

foo1(x) = minimum(x)

function foo2(x)
    output = Inf

    for i in eachindex(x)
        output = min(output, x[i])
    end

    return output
end
```

```
julia> foo1(x)
0.0303789
julia> foo2(x)
0.0303789
```

AVOIDING MEMORY ALLOCATIONS THROUGH REDUCTIONS

One of the key advantages of reductions is that they avoid unnecessary memory allocation, especially for intermediate results.

To illustrate, consider the operation `sum(log.(x))` for a vector `x`. Its computation involves two steps: transforming `x` by applying `log` element-wise, and summing the transformed elements. Since broadcasting computes its results **eagerly**, it implies the internal creation of a new vector to store the

values of `log.(x)`. Consequently, this step results in memory allocations.

In many cases like this, however, only the scalar output matters, with the intermediate result being of no interest. In this context, computational strategies that obtain the final output while bypassing the allocation of intermediate results like `log.(x)` are preferred.

Reductions make this possible by defining a scalar `output`, which is iteratively updated by summing the transformed values of `x`. In our example, this means that each element of `x` is transformed by the logarithm, and the result is then immediately added to the accumulator. In this way, the storage of the intermediate vector `log.(x)` is entirely avoided.

SUM

```
x       = rand(100)

foo1(x) = sum(log.(x))

function foo2(x)
    output = 0.0

    for i in eachindex(x)
        output += log(x[i])
    end

    return output
end
```

```
julia> @btime foo1($x)
  719.923 ns (2 allocations: 928 bytes)
julia> @btime foo2($x)
  706.610 ns (0 allocations: 0 bytes)
```

PRODUCT

```
x       = rand(100)

foo1(x) = prod(log.(x))

function foo2(x)
    output = 1.0

    for i in eachindex(x)
        output *= log(x[i])
    end

    return output
end
```

```
julia> @btime foo1($x)
  753.135 ns (2 allocations: 928 bytes)
julia> @btime foo2($x)
  727.667 ns (0 allocations: 0 bytes)
```

MAXIMUM

```
x       = rand(100)

foo1(x) = maximum(log.(x))

function foo2(x)
    output = -Inf

    for i in eachindex(x)
        output = max(output, log(x[i]))
    end

    return output
end
```

```
julia> @btime foo1($x)
  1.218 μs (2 allocations: 928 bytes)
julia> @btime foo2($x)
  948.161 ns (0 allocations: 0 bytes)
```

MINIMUM

```
x       = rand(100)

foo1(x) = minimum(log.(x))

function foo2(x)
    output = Inf

    for i in eachindex(x)
        output = min(output, log(x[i]))
    end

    return output
end
```

```
julia> @btime foo1($x)
  1.295 μs (2 allocations: 928 bytes)
julia> @btime foo2($x)
  981.767 ns (0 allocations: 0 bytes)
```

REDUCTIONS VIA BUILT-IN FUNCTIONS

So far, we've manually implemented reductions with intermediate transformations through explicit for-loops. While this approach makes the underlying mechanics transparent, it also introduces considerable verbosity.

To avoid this inconvenience, Julia provides several streamlined alternatives for expressing reductions. One such alternative is through specialized methods of common reduction functions, including `sum`, `prod`, `maximum`, and `minimum`. These methods accept a transforming function as their first argument, with the collection to be reduced as a second argument. The general syntax is `foo(<transforming function>, x)`, where `foo` is the reduction function and `x` is the vector to be transformed and reduced.

The following examples illustrate this approach by applying a logarithmic transformation prior to the reduction.

SUM

```
x      = rand(100)

foo(x) = sum(log, x)        #same output as sum(log.(x))
```

```
julia> @btime foo($x)
  799.583 ns (0 allocations: 0 bytes)
```

PRODUCT

```
x      = rand(100)

foo(x) = prod(log, x)       #same output as prod(log.(x))
```

```
julia> @btime foo($x)
  768.722 ns (0 allocations: 0 bytes)
```

MAXIMUM

```
x      = rand(100)

foo(x) = maximum(log, x)    #same output as maximum(log.(x))
```

```
julia> @btime foo($x)
  1.363 μs (0 allocations: 0 bytes)
```

MINIMUM

```
x      = rand(100)

foo(x) = minimum(log, x)    #same output as minimum(log.(x))
```

```
julia> @btime foo($x)
  1.375 μs (0 allocations: 0 bytes)
```

The specialized function methods are commonly applied using anonymous functions, as shown below.

SUM

```
x      = rand(100)

foo(x) = sum(a -> 2 * a, x)        #same output as sum(2 .* x)
```

```
julia> @btime foo($x)
  17.333 ns (0 allocations: 0 bytes)
```

PRODUCT

```
x      = rand(100)

foo(x) = prod(a -> 2 * a, x)       #same output as prod(2 .* x)
```

```
julia> @btime foo($x)
  20.968 ns (0 allocations: 0 bytes)
```

MAXIMUM

```
x      = rand(100)

foo(x) = maximum(a -> 2 * a, x)    #same output as maximum(2 .* x)
```

```
julia> @btime foo($x)
  406.685 ns (0 allocations: 0 bytes)
```

MINIMUM

```
x      = rand(100)

foo(x) = minimum(a -> 2 * a, x)    #same output as minimum(2 .* x)
```

```
julia> @btime foo($x)
  415.786 ns (0 allocations: 0 bytes)
```

These specialized methods also accept transforming functions with multiple arguments. In this case, the arguments must be paired using `zip`, with indices corresponding to each argument within the transforming function. This is illustrated below, with the transforming operation `x .* y`.

SUM

```
x        = rand(100)
y        = rand(100)

foo(x,y) = sum(a -> a[1] * a[2], zip(x,y))          #same output as sum(x .* y)
```

```
julia> @btime foo($x)
  78.454 ns (0 allocations: 0 bytes)
```

PRODUCT

```
x        = rand(100)
y        = rand(100)

foo(x,y) = prod(a -> a[1] * a[2], zip(x,y))        #same output as prod(x .* y)
```

```
julia> @btime foo($x)
  94.980 ns (0 allocations: 0 bytes)
```

MAXIMUM

```
x        = rand(100)
y        = rand(100)

foo(x,y) = maximum(a -> a[1] * a[2], zip(x,y))     #same output as maximum(x .* y)
```

```
julia> @btime foo($x)
  385.753 ns (0 allocations: 0 bytes)
```

MINIMUM

```
x        = rand(100)
y        = rand(100)

foo(x,y) = minimum(a -> a[1] * a[2], zip(x,y))     #same output as minimum(x .* y)
```

```
julia> @btime foo($x)
  380.757 ns (0 allocations: 0 bytes)
```

THE "REDUCE" AND "MAPREDUCE" FUNCTIONS

Beyond the specific cases discussed, reductions are applicable whenever a binary operation meets the necessary conditions for their implementation. To accommodate this generality, Julia provides the functions `reduce` and `mapreduce`.[2]

The `reduce` function applies a binary operation directly to the elements of a collection, combining them into a single result. By contrast, `mapreduce` transforms each element before applying the reduction.

It's worth remarking that reductions for sums, products, maximum, and minimum should still be implemented via their dedicated functions. This is because the specialized methods in `sum`, `prod`, `maximum`, and `minimum` have been carefully optimized for their respective tasks, typically outperforming `reduce` and `mapreduce`.

FUNCTION "REDUCE"

The function `reduce` uses the syntax `reduce(<function>,x)`, where `<function>` is a two-argument function.

SUM

```
x      = rand(100)

foo(x) = reduce(+, x)           #same output as sum(x)
```

```
julia> @btime foo($x)
  15.738 ns (0 allocations: 0 bytes)
```

PRODUCT

```
x      = rand(100)

foo(x) = reduce(*, x)           #same output as prod(x)
```

```
julia> @btime foo($x)
  16.330 ns (0 allocations: 0 bytes)
```

MAXIMUM

```
x      = rand(100)

foo(x) = reduce(max, x)         #same output as maximum(x)
```

```
julia> @btime foo($x)
  379.038 ns (0 allocations: 0 bytes)
```

MINIMUM

```
x      = rand(100)

foo(x) = reduce(min, x)         #same output as minimum(x)
```

```
julia> @btime foo($x)
  361.919 ns (0 allocations: 0 bytes)
```

FUNCTION "MAPREDUCE"

The `mapreduce` function integrates `map` and `reduce` into a unified operation, explaining its name. Thus, it applies a transformation via the function `map` before doing the reduction. Its syntax is `mapreduce(<transformation>,<reduction>,x)`. To illustrate its use, we consider a `log` transformation.

SUM

```
x      = rand(100)

foo(x) = mapreduce(log, +, x)       #same output as sum(log.(x))
```

```
julia> @btime foo($x)
  700.262 ns (0 allocations: 0 bytes)
```

PRODUCT

```
x       = rand(100)

foo(x) = mapreduce(log, *, x)        #same output as prod(log.(x))
```

```
julia> @btime foo($x)
  732.325 ns (0 allocations: 0 bytes)
```

MAXIMUM

```
x       = rand(100)

foo(x) = mapreduce(log, max, x)      #same output as maximum(log.(x))
```

```
julia> @btime foo($x)
  1.244 μs (0 allocations: 0 bytes)
```

MINIMUM

```
x       = rand(100)

foo(x) = mapreduce(log, min, x)      #same output as minimum(log.(x))
```

```
julia> @btime foo($x)
  1.256 μs (0 allocations: 0 bytes)
```

`mapreduce` can also be used with anonymous functions and functions with multiple arguments. Below, we illustrate these possibilities.

SUM

```
x         = rand(100)
y         = rand(100)

foo(x,y) = mapreduce(a -> a[1] * a[2], +, zip(x,y))        #same output as sum(x .* y)
```

```
julia> @btime foo($x)
  75.167 ns (0 allocations: 0 bytes)
```

PRODUCT

```
x         = rand(100)
y         = rand(100)

foo(x,y) = mapreduce(a -> a[1] * a[2], *, zip(x,y))        #same output as prod(x .* y)
```

```
julia> @btime foo($x)
  89.958 ns (0 allocations: 0 bytes)
```

MAXIMUM

```
x        = rand(100)
y        = rand(100)

foo(x,y) = mapreduce(a -> a[1] * a[2], max, zip(x,y))     #same output as maximum(x .* y)
```

```
julia> @btime foo($x)
  370.026 ns (0 allocations: 0 bytes)
```

MINIMUM

```
x        = rand(100)
y        = rand(100)

foo(x,y) = mapreduce(a -> a[1] * a[2], min, zip(x,y))     #same output as minimum(x .* y)
```

```
julia> @btime foo($x)
  365.562 ns (0 allocations: 0 bytes)
```

REDUCE OR MAPREDUCE?

`mapreduce(<transformation>,<operator>,x)` yields the same result as `reduce(<operator>, map(<transformation>,x))`. Despite this, `mapreduce` should be employed if the vector input must be transformed beforehand. The reason is that `mapreduce` avoids the internal memory allocations of the transformed vector, while `map` doesn't. This aspect is demonstrated below, where `sum(2 .* x)` is computed through a reduction.

MAPREDUCE

```
x      = rand(100)

foo(x) = mapreduce(a -> 2 * a, +, x)
```

```
julia> @btime foo($x)
  15.887 ns (0 allocations: 0 bytes)
```

REDUCE

```
x      = rand(100)

foo(x) = reduce(+, map(a -> 2 * a, x))
```

```
julia> @btime foo($x)
  82.306 ns (2 allocations: 928 bytes)
```

FOOTNOTES

[1.] Strictly speaking, an identity element isn't always required. In many cases, no natural identity exists, yet the first element of the collection can serve as the initial value. For simplicity, however, we assume the existence of an identity element as a requirement.

[2.] Both functions are convenient for packages that implement specialized versions of reductions. By simply defining these two functions, the package is capable of covering all possible reductions.